

FITBUDDY

AI FITNESS PLAN GENERATOR USING GEMINI MODELS

Detailed Project Documentation

Project Overview

FitBuddy is an AI-powered personalized fitness planning web application designed to generate structured workout plans from user-provided fitness information. The application combines a FastAPI backend, Jinja2-based web pages, SQLite database storage, and Gemini models to produce individualized fitness guidance.

The documentation below is a detailed 12-page version of the supplied FitBuddy project material. It preserves the project's main terminology and workflow while providing additional explanation for academic/project-report use.

Project Area	Technology / Component
Backend	Python, FastAPI, Uvicorn
AI Integration	Gemini models configured through environment variables
Database	SQLite with SQLAlchemy
Frontend	HTML, CSS, JavaScript, Jinja2 templates
Main Purpose	Personalized workout generation, feedback collection, nutrition/recovery guidance

Prepared as: College Project Documentation

1. Project Description and Objectives

FitBuddy is developed as a personalized fitness planning application. Instead of presenting the same workout routine to every user, the system collects basic profile and fitness-goal information and sends that context to an AI generation layer. The generated result is then displayed through a web interface and stored for later access.

The supplied project material describes the system as supporting workout-plan generation, nutrition and recovery tips, feedback submission, adaptive plan updates, and administrative/user-management functions.

Project objectives

- Create an easy-to-use web interface where users can enter their fitness information.
- Generate a personalized workout plan using a Gemini model through the FastAPI backend.
- Provide supporting nutrition and recovery guidance as part of the generated fitness experience.
- Store user information, generated plans, feedback, and timestamps in the application database.
- Allow submitted feedback to be used when updating an existing workout plan.
- Provide API and administrative routes for viewing, retrieving, and deleting user records.

Main user scenarios

Scenario	Detailed Flow
Plan Generation	User enters profile details → backend validates/receives data → Gemini generation function creates a plan → result is displayed and stored.
Feedback Update	User submits feedback → backend receives the feedback → existing plan can be updated using the feedback context → updated information is stored.
Nutrition / Recovery	The AI layer can provide nutrition or recovery-oriented guidance alongside the workout information.
Administration	Administrative routes support viewing users, retrieving individual user information, and deleting records when required.

2. Technical Architecture

The application follows a layered web-application structure. The browser handles the user-facing pages, while FastAPI manages requests, validation, business logic, database operations, and communication with the Gemini generation functions.

Layer	Component	Responsibility
Presentation	HTML / CSS / JavaScript / Jinja2	Collect input, display forms, results, feedback and user information.
Application	FastAPI routes	Receive requests, process form/API data and coordinate application logic.
AI Service	Gemini generator functions	Generate workout plans, nutrition tips and feedback-based updates.
Data Layer	SQLAlchemy + SQLite	Persist users, plans, feedback, nutrition information and timestamps.
Server	Uvicorn	Run the FastAPI application locally or in a deployment environment.

Request and response flow

- Step 1 – The user opens the FitBuddy home page and enters profile information such as username, user ID, age, weight and fitness goal.
- Step 2 – The frontend sends the submitted information to the appropriate FastAPI route.
- Step 3 – The backend prepares the request context and calls the appropriate Gemini generation function.
- Step 4 – The AI-generated workout information is returned to the application.
- Step 5 – The application stores the relevant information in SQLite and prepares the result page.
- Step 6 – Jinja2 renders the response as an HTML page that the user can read in the browser.

This architecture separates presentation, backend logic, AI generation, and data persistence, making the project easier to organize and maintain.

3. Development Workflow and Project Milestones

The supplied documentation identifies a sequence of development milestones covering model selection, application architecture, backend implementation, frontend development, and deployment/testing.

Milestone	Work Completed / Purpose
Model Selection & Architecture	Define how Gemini models, FastAPI, database storage and web templates work together.
Core Functionality	Implement user input handling and the main workout-generation workflow.
routes.py Development	Create application routes for generation, feedback, user access, deletion, APIs and health checking.
Frontend Development	Build user-facing pages and connect them to backend routes using forms and templates.
Deployment & Testing	Run the application through Uvicorn, test the major workflows and verify route/database behavior.

Typical development cycle

- Design – identify required pages, data fields, AI functions and database information.
- Implement – write FastAPI routes, database models, Gemini generator functions and frontend templates.
- Integrate – connect forms and templates to the backend routes.
- Test – verify successful generation, database storage, feedback submission and user retrieval.
- Refine – improve prompts, page layout, validation and error handling based on testing.

The project structure is intended to keep reusable backend functionality separated into modules such as database handling, routes and Gemini generator files.

4. AI / Gemini Model Integration

The AI component is the central feature of FitBuddy. The supplied source code shows separate generator functions for workout generation, nutrition-tip generation, and feedback-based workout-plan updates. Model names are configurable through environment variables rather than being hard-coded into the documentation.

Workout generation

- The workout generator receives user-specific fitness context such as age, weight, goal and workout preferences.
- The backend prepares a structured prompt describing the required output.
- The Gemini model generates a personalized workout response.
- The generated response is returned to the application and presented on the result page.

Nutrition and recovery guidance

The project also includes a Gemini Flash-based generation function for nutrition/recovery-oriented tips. This separates the supporting guidance task from the main workout-plan generation workflow.

Feedback-based adaptation

FitBuddy includes an update function that uses submitted feedback as additional context. The intended workflow is to take the previous plan, combine it with the user's feedback, and request an updated plan from the AI model.

Safety-oriented prompt design

The supplied code includes general-wellness guidance in the AI prompt and avoids positioning the system as a medical diagnosis or treatment tool. The documentation therefore describes the generated output as general fitness guidance rather than medical advice.

5. Backend and Database Design

The backend is implemented with Python and FastAPI. SQLAlchemy is used as the database interaction layer, with SQLite identified in the supplied project configuration as the local database.

Stored information

Data	Purpose
User ID	Identifies the user record and supports user-specific retrieval/update operations.
User profile	Stores relevant profile values such as username, age, weight and fitness goal.
Original plan	Stores the initial AI-generated workout information.
Updated plan	Stores the revised plan when feedback-based adaptation is performed.
Nutrition tip	Stores supporting nutrition/recovery guidance generated for the user.
Feedback	Stores user comments that can support later plan updates.
Timestamp	Provides a record of when relevant information was created or updated.

Database workflow

- A request is received by the FastAPI route.
- The backend validates or processes the supplied values.
- AI output and relevant user information are associated with the user record.
- SQLAlchemy handles database operations against SQLite.
- The stored information can later be accessed through web pages or API endpoints.

This persistence layer is important because the application is not limited to one-time AI output; it keeps information that supports later viewing and feedback-based updates.

6. FastAPI Routes and Application Logic

The supplied project documentation identifies several routes that form the main interface between the frontend, backend services and stored data.

Route	Function
/	Loads the main FitBuddy interface where the user can enter fitness information.
/generate-workout	Processes workout-generation input and invokes the AI generation workflow.
/submit-feedback	Receives feedback submitted by a user and supports the feedback/update workflow.
/view-all-users	Provides an administrative view of stored users.
/delete-user/{user_id}	Deletes a specified user record using the supplied user ID.
/api/users	Provides access to user information through an API endpoint.
/api/users/{user_id}	Provides user-specific API access using the user ID.
/health	Returns a health status response used to check whether the application is running.

Application startup

The FastAPI application creates the application instance, mounts static files, includes the project router, and initializes the database during startup. The project is intended to be run through Uvicorn during local development.

The health endpoint is especially useful during development because it provides a simple way to verify that the FastAPI server is responding before testing more complex AI and database workflows.

7. Frontend Development and Templates

The frontend provides the interaction layer for FitBuddy. The supplied material references Jinja2 templates and pages such as the main input page, the result page and the all-users page.

Main frontend components

Component	Purpose
index.html	Main input interface for collecting user fitness details and starting plan generation.
result.html	Displays the personalized plan, user information and generated fitness content.
all_users.html	Displays stored user information for the administrative workflow.
Feedback page/form	Collects user ID and written feedback for the adaptive planning workflow.
CSS / static assets	Controls visual presentation and supporting browser-side assets.
JavaScript	Supports browser interactions such as button actions and form-related behavior.

Result page content

The supplied result template includes a personalized greeting, user ID, age, weight and fitness goal, together with the AI-generated fitness plan. This creates a direct connection between the information entered by the user and the generated result.

Frontend-to-backend connection

- The user fills in the form fields.
- The form submits data to the relevant FastAPI route.
- The backend performs processing and AI generation.
- The result is passed into a Jinja2 template.
- The browser displays the generated plan in a structured page.

8. User Interface – Home Page and Plan Generation

The following screenshot is taken from the supplied FitBuddy project documentation. It shows the main user interface used to collect the information required for personalized plan generation.

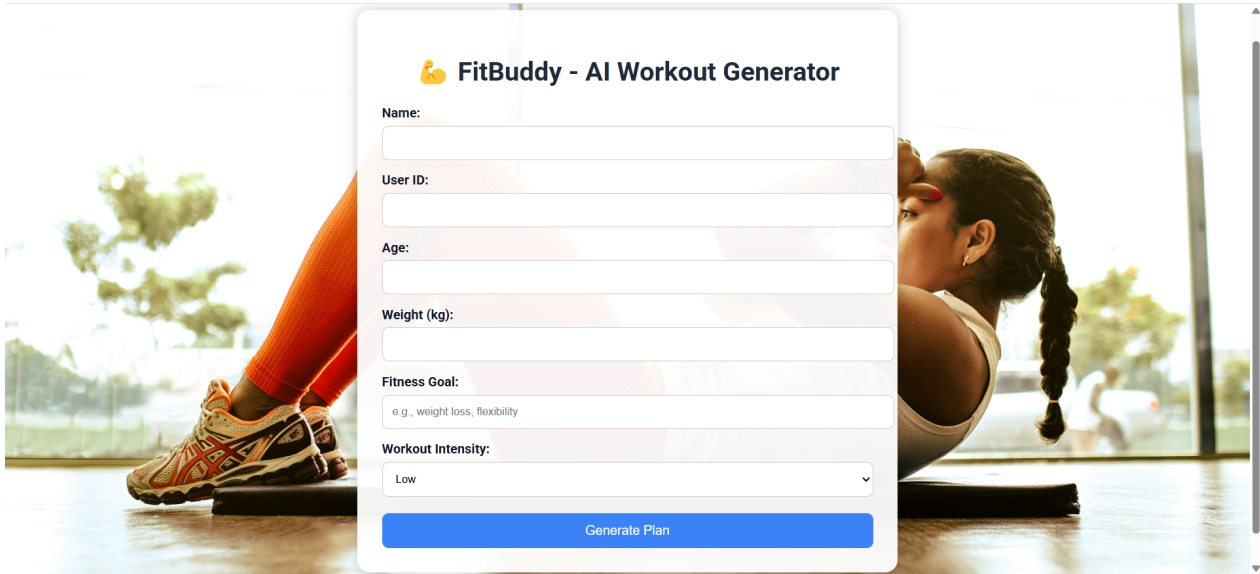
A screenshot of the FitBuddy - AI Workout Generator interface. The interface is a white card with rounded corners, centered over a background image of a person's legs in orange socks and sneakers, and a person's head in profile. The card has a title 'FitBuddy - AI Workout Generator' with a small orange icon. Below the title are several input fields: 'Name:', 'User ID:', 'Age:', 'Weight (kg):', 'Fitness Goal:' (with a placeholder 'e.g., weight loss, flexibility'), and 'Workout Intensity:' (with a dropdown menu showing 'Low'). At the bottom of the card is a blue button labeled 'Generate Plan'.

Figure 1. FitBuddy home page / AI workout generator interface.

Interface fields shown

- **Name:** identifies the user in the application interface.
- **User ID:** provides a record identifier used by the backend.
- **Age:** supplies an age value used as part of the personalization context.
- **Weight:** supplies the user's weight information in kilograms.
- **Fitness Goal:** describes the user's intended fitness objective.
- **Workout Intensity:** allows the user to specify the desired intensity level.
- **Generate Plan:** starts the workout-generation workflow.

After submission, the backend receives the form information and uses it as context for AI-generated fitness planning. The result is then displayed through the application's result interface.

9. Generated Plan and Feedback Workflow

FitBuddy is designed not only to generate an initial plan but also to support an iterative interaction through feedback. The project documentation includes interface screenshots showing the feedback workflow.

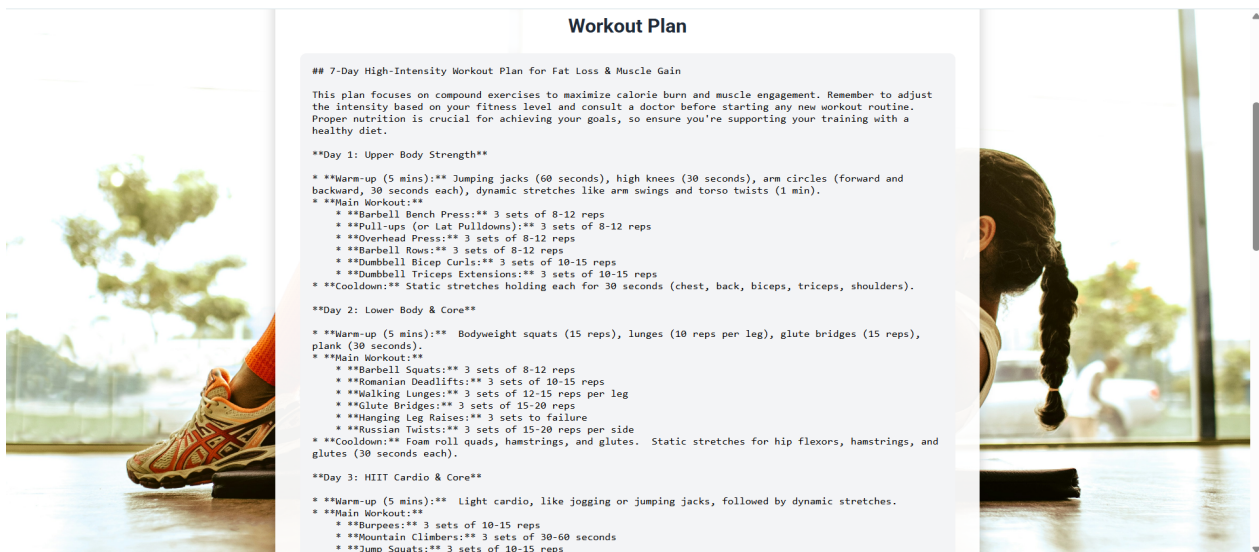


Figure 2. FitBuddy generated-plan interface screenshot from the supplied project material.

The image shows a screenshot of a web application titled "Share Your Feedback". The form includes a section for "Your Unique User ID" with a text input field, a section for "Your Feedback" with a larger text area, and a blue "Submit Feedback" button at the bottom.

Figure 3. FitBuddy feedback submission interface.

Feedback process

- The user identifies the relevant record using the User ID.
- The user enters comments describing their experience or required changes.
- The feedback is submitted to the backend through the feedback route.
- The application can use the existing plan and the new feedback as context for an updated plan.
- The updated information can be stored with the user record for later reference.

This workflow gives the application a simple feedback loop: **Generate** → **Use** → **Give Feedback** → **Update**. It is one of the main features that distinguishes the application from a static fitness-information webpage.

10. Setup, Testing and Local Deployment

The project is intended to be run as a Python FastAPI application. During development, Uvicorn acts as the application server and can be started in reload mode so that code changes are detected automatically.

Recommended setup sequence

- Create or activate the project's Python virtual environment.
- Install the packages listed in the project requirements file.
- Ensure the expected backend and static/template directories exist.
- Configure required environment variables, including the Gemini API credential, without placing secrets directly in source code.
- Start the FastAPI application through Uvicorn.
- Open the local application URL in a browser.
- Use the /health endpoint to confirm that the server is responding.

Testing checklist

Test Area	Expected Verification
Home page	Form loads correctly and required fields are visible.
Plan generation	Submitted user data reaches the backend and a generated result is returned.
Database	User and plan information is stored and can be retrieved.
Feedback	Feedback can be submitted with the correct user ID.
User APIs	User listing and user-specific endpoints respond correctly.
Delete route	Specified user record can be removed when the administrative function is used.
Health check	Endpoint returns a healthy application status.

Security note

API keys and other secrets should be supplied through environment variables or secure configuration. They should not be included in public documentation, screenshots, source repositories or shared project files.

11. Conclusion and Future Scope

FitBuddy demonstrates how an AI model can be integrated into a full-stack web application to create personalized fitness planning. The project combines a browser-based interface with FastAPI routes, database persistence and Gemini-powered generation functions.

Key achievements

- Implemented a web-based input workflow for collecting user fitness information.
- Integrated AI generation into the backend application flow.
- Created a persistent database layer for user and plan information.
- Provided a feedback mechanism for iterative plan updates.
- Added API and administrative routes for user management.
- Created separate frontend templates for input, results and user information.
- Included a health-check endpoint for basic service verification.

Future scope

- Improve personalization by using additional fitness preferences and activity information.
- Add authentication and role-based access for users and administrators.
- Improve validation and error handling for invalid or incomplete input.
- Add richer progress tracking and historical plan comparison.
- Provide more detailed dashboards for users to monitor their plans and feedback.
- Extend deployment beyond local development with secure cloud configuration.

Final summary

The completed FitBuddy architecture provides a clear foundation for an AI-assisted fitness planning system. Its modular backend, database storage, web templates and configurable Gemini integration make it suitable as a college-level full-stack and artificial-intelligence project demonstration.

Module	Main Technology	Purpose
Frontend	HTML/CSS/JavaScript/Jinja2	User interaction and presentation
Backend	Python/FastAPI	Application and route processing
AI	Gemini models	Personalized generation and updates
Database	SQLite/SQLAlchemy	Persistent project data
Server	Uvicorn	Local application execution

End of FitBuddy Project Documentation